ChatGPT

Systematic Audit and Consolidation Roadmap for concept-research-scout

Executive summary

I audited the August 24 repository snapshot as code, not just the planning packet. I unpacked and statically inspected the current tree; examined `scout.py`, `orchestrator/ledger.py`, `AGENTS.toml`, the six GitHub Actions workflows, current state and ledger materializations, Idea 023's contract/code/notebook, result validation, prompts, tests, and accumulated legacy paths; ran setup/compile/doctor checks; and attempted the full 108-test orchestration suite. I also compared the architecture with the current public Co-Scientist, IDEAgent, ResearchAgent, and Google's published Co-Scientist system using primary project/paper sources. Claude's Round 4 packet is an accurate description of the intended next phase, but the code audit changes the priority order in several important ways. filecite turn0file0

My overall conclusion is:

> **The research system is now genuinely useful and scientifically interesting. Its core problem is no longer lack of capability; it is that the execution/governance infrastructure has not fully caught up with the sophistication of the research workflow.**

The central scientific architecture has earned its existence: keystone screening, explicit failure semantics, debate/revision, claim retention, feasibility, hash-bound contracts, cross-family probe review, deterministic approval binding, result interpretation, and conservative human ratification. Idea 004 demonstrated the whole principle. Idea 023 is an even stronger test because the workflow is now handling an actual multi-phase, contract-amended, real-data experiment rather than a single fixed probe. The Round 4 packet accurately describes that progression through synthetic Phase S, deterministic contract amendment, renewed approval, multiple code-review rounds, and the Phase C census. filecite turn0file0

However, I found one **immediate Idea-023 execution-path incompatibility** that should be resolved before treating the running census as safely recordable:

> **The committed generic results validator and generated Colab launcher do not currently agree with Idea 023's Phase S/C protocol.**

Specifically:

- `validate_bundle()` accepts only phases `M` and `B` and universally requires `manifest/pair_manifest.csv` (`scout.py:1082–1141`).
- Idea 023 produces phase `"C"` and its contract does not require `manifest/pair_manifest.csv`; it instead requires an archive manifest, split manifest, schema census, per-patient results, and other

census-specific outputs (`probes/023/run.py:742–747` ; `ideas/023/probe_contract.yaml:122–137`).
- `results-validate.yml` likewise hard-codes `probes/$IDEA/results_v2` and M/B completion semantics (`.github/workflows/results-validate.yml:53–104`).
- More urgently, `probes/023/run.py` refuses Phase C unless `--phase-s-dir` is supplied (`run.py:300–303`), but the committed generated Phase-C notebook invokes `run.py` without that argument. The generator itself constructs the command without `--phase-s-dir` (`scout.py:1011–1014`).

Therefore, **the committed notebook cannot reproduce a successful Phase-C launch as written**. If the Colab job is genuinely running now, the live invocation must differ from the committed notebook or have been manually repaired. That is not necessarily a scientific problem—the correct live execution may be perfectly valid—but it is an audit-trail problem and needs to be captured immediately.

This also means the current freeze list is paradoxically too strict: it freezes `results-validate` even though that validator cannot ingest the 023 result bundle as currently defined. I would preserve the scientific freeze on `ideas/023/` , `probes/023/run.py` , the contract, and the running data analysis, but explicitly permit a **deterministic transport/validator compatibility patch** that is tested against a synthetic Idea-023 Phase-C bundle before the real result push. The plan's current freeze list should therefore be amended rather than followed literally. filecite turn0file0

Beyond that immediate issue, my recommended strategy is:

> **Finish the 023 result safely → fix run/state synchronization and provenance → build governance/security receipts → evaluate the system prospectively → modularize and simplify → only then expand autonomy.**

I would **not** make the blackboard arbiter, proposal meta-loop, third charter, Evolution, or additional autonomous processes the next engineering focus.

A rough current assessment:

Dimension	Assessment
Scientific epistemology	**9/10**
Cheap falsification and negative-result handling	**9/10**
Experiment contract discipline	**9/10**
Human-gated research governance concept	**9/10**
Actual end-to-end usefulness	**High; demonstrated**
Multi-charter correctness	**~7/10 after recent repairs**
Result plumbing generality	**~5/10; 004 assumptions still leak into 023**
Provenance completeness	**~5/10**

Dimension	Assessment
Runtime/state architecture	**~5/10**
Git/CI robustness	**~5/10**
Security privilege separation	**~4–5/10**
Evaluation of agent/stage value	**~4/10**
Maintainability	**~6/10**
Need for more agent roles	**Low**

The main message for tomorrow's work with Claude is therefore not “redesign the scout.” It is:

The research design is ahead of the software substrate. Consolidate the substrate now.

Audit evidence and current system health

The snapshot contains a substantial system rather than a prompt collection. `scout.py` is now approximately **2,849 lines**, `orchestrator/ledger.py` 384 lines, and the main orchestration test file 2,402 lines. There are six GitHub Actions workflows. The live ledger materialization contains **161 current logical entries**, plus six historical invalid-row tombstones when explicitly requested. There are **44 numbered ideas**; 40 have reached `DEBATED` scrutiny, one has a `PROBED` scrutiny state, and the ranked candidate inventories currently contain about **64 baseline and 16 ISLES candidates**. That last number is important: the system currently has a large idea inventory relative to its experiment throughput.

Basic static checks are healthy:

Check	Result
`python setup.py`	Pass
Python compilation of main orchestrator/ledger	Pass
`python scout.py doctor`	Pass
Family-opposition checks	Pass
Charter counters/resolution doctor checks	Pass
Ledger current-state load	Pass
Tombstones excluded by default	Pass

The latest tombstone/charter closeout work remains sound. `charter_for_target()` now resolves a persisted target from the target itself and fails closed on malformed cards (`scout.py:104-133`). `ledger.load()` has the correct current-state versus audit-history distinction. `ledger.jsonl` is no longer union-merged. Per-charter digests and portfolio briefs exist, and the factual `cross_charter_index.md` is injected separately. Those are meaningful improvements over the earlier

architecture and correctly encode the principle that persistent target identity outranks mutable global cycle state. The earlier report's multi-charter corruption and shared-state incidents were real production failures rather than hypothetical concerns, so prioritizing these repairs was correct. filecite turn0file1

Independent test-suite result

The Round 4 packet reports **108 tests passing** and treats the suite as the merge gate. filecite turn0file0 I independently counted 108 test methods, but I could **not reproduce a complete full-suite pass within ten minutes** in this audit environment.

`python -m unittest -v tests.test_orchestration` progressed through roughly the first several dozen tests without a failure and then exceeded the ten-minute execution window while around `TestCycle.test_fiction_cycle_is_blind_and_merges`. Running that particular test alone completed successfully in about four seconds. Several individual classes and targeted tests also passed.

That pattern points more toward **suite-level cumulative cost or order-dependent slowdown** than a deterministic failure in that test. It does not prove there is a hang, and GitHub's environment may complete the suite normally, but I would no longer describe test scaling as entirely solved.

The fixture optimization has clearly worked: the test harness now constructs a small reusable fixture rather than repeatedly copying the historical research corpus. The remaining cost is more likely repeated Git/subprocess/orchestration work.

I would split the merge gate conceptually into:

```
fast deterministic unit/invariant suite
        ↓
Git/repository integration suite
        ↓
slow packaging/workflow integration suite
```

with a hard runtime budget and timing report for each. A hard scientific merge gate that takes long enough that developers avoid running it locally eventually stops being a useful gate.

The Idea-023 path needs attention before its result lands

This is the most important concrete finding of the current audit.

The Round 4 packet says Phase C is running against the amended contract and that results will flow onto `results/probe-023-349af5ad0b3e`. filecite turn0file0 The committed execution infrastructure does not yet describe that path coherently.

The incompatibilities are:

Component	Current assumption	Idea 023 reality
`validate_bundle()`	phase ∈ `{M,B}`	phase `C`
`validate_bundle()`	`manifest/pair_manifest.csv` universally required	no pair manifest in 023 contract
results workflow	bundle always `results_v2`	still used, but now semantically meaningless version name
results completion	Phase M or B-analysis	Phase C census
package notebook	no Phase-S dependency argument	Phase C requires `--phase-s-dir`
contract outputs	validator's fixed five-file skeleton	023 has a different required-output set

The problem is architectural, not merely a 023 special case:

> `validate_bundle()` currently contains an **Idea-004 result ontology disguised as a universal result schema**.

The planned Patch 2c idea that `package-colab` should read phase vocabulary and staging specification from the probe contract is therefore right, but part of that generalization has become **P0 for 023**, not future cleanup. filecite turn0file0

A better interface would look approximately like:

```
@dataclass(frozen=True)
class ResultSpec:
    allowed_phases: frozenset[str]
    completion_phases: frozenset[str]
    required_outputs: tuple[str, ...]
    manifest_checks: tuple[ManifestCheck, ...]


def result_spec(idea: int) -> ResultSpec:
    # Prefer a machine-readable block in the frozen probe registry/contract.
    ...


def validate_bundle(idea: int, bundle: Path) -> list[str]:
    spec = result_spec(idea)

    for rel in spec.required_outputs:
        require_regular_file_within(bundle, rel)
```

```
    summary = load_json(bundle / "summary.json")

    if summary["phase"] not in spec.allowed_phases:
        ...

    verify_contract_binding(...)
    verify_declared_manifests(...)
```

Do **not** solve this with:

```
if idea == 23:
    allow_phase_c = True
```

That would merely add another sediment layer.

I would also immediately capture the exact Phase-C command that is actually running in Colab. If it contains the missing:

```
--phase-s-dir ...
```

record that command, Phase-S directory hash, notebook commit, contract hash, and environment as an execution receipt. If it does not, then the committed `run.py` should have refused before touching the real census.

Issue register

The table below separates current correctness issues from architectural debt. File/line references refer to the supplied snapshot.

Severity	Finding	Evidence	Remediation
High	**Idea-023 result plumbing is incompatible with its frozen S/C protocol**	`scout.py:1082–1141`; `.github/workflows/results-validate.yml:53–104`; `probes/023/run.py:742–747`; contract `required_outputs` around `ideas/023/probe_contract.yaml:122–137`	Before final result push, make result validation contract/registry-driven and test a synthetic Phase-C bundle. Amend freeze to permit this transport-only fix.

Severity	Finding	Evidence	Remediation
High	**Committed Phase-C notebook omits required `--phase-s-dir`**	`probes/023/run.py:300–303`; generator `scout.py:1011–1014`; committed notebook's execution cell	Capture actual running Colab command immediately. Update generic package schema after the frozen run. If execution was manually corrected, add the correction to provenance rather than rewriting history.
High	**Shared mutable `state.json` still sits on the critical path after two actual Git-race incidents**	`scout.py:37–45`; Round 4 incident history filecite turn0file0	Patch 2a should separate immutable run events from materialized per-idea scientific state. Treat generated state as cache/view, not authority.
High	**Git rebase failure is still ignored before retrying a push**	`_push_checkpoint`, `scout.py:1663–1676`; workflow rescue blocks such as `idea-pipeline.yml:102–108`, `actioner.yml:82–89`	Check every fetch/rebase return code. On conflict: `git rebase --abort`, preserve logs, fail job. Never commit/push a conflicted worktree.
High	**Stage provenance is incomplete by construction**	`run_agent()` `391–462`; `_record_stage_provenance()` `2369–2385`; debate returns before recorder `2390–2395`; custom probe/interpret paths call `run_agent()` directly	Persist an execution receipt **inside** `run_agent()` for every attempt, then a separate promotion/stage-success event after deterministic validation. Retire `LAST_RUN` as provenance authority.
High	**Agent jobs have repo-write capability while model code executes**	`idea-pipeline.yml:39–41`, `actioner.yml:20–22`; checkout defaults; `AGENTS.toml:45,50,56`	Agent step gets read checkout with `persist-credentials:false` and no repository-write token. A later deterministic step performs scoped commit/push.

Severity	Finding	Evidence	Remediation
High	**Ledger's current-state semantics remain order-sensitive under concurrent same-ID events**	`orchestrator/ledger.py:59–118`: append-only records + latest physical record wins	Add immutable event IDs, causal/source revision, deterministic ordering and explicit conflict detection. Do not let Git line order decide conflicting scientific decisions.
Medium	**Charters are identity-scoped but not yet true policy bundles**	`build_prompt()`, `scout.py:197–201`, always injects global `docs/SCORING_RUBRIC.md`; generation prompts/schema remain globally interpretability-shaped	Move generation policy, rubric and candidate schema into charter profiles. Keep keystone/debate/contract discipline shared.
Medium	**A new named charter can still fall back to the global digest**	`_digest_path()`, `scout.py:171–181`	Mirror the corrected portfolio-brief behavior: named charter with no digest gets empty/scoped digest, never global fallback.
Medium	**Rotation is still derived from mutable global `active_cycle` when cycle is not explicit**	`effective_agent()`, `scout.py:240–253`	Freeze a role epoch per idea-stage/run and pass it explicitly. Patch 2a already proposes this and should keep it.
Medium	**Result-manifest path handling lacks containment checks**	`validate_bundle()`, `scout.py:1129–1140`	Resolve every manifest path and reject traversal, symlinks, directories and anything outside bundle root.
Medium	**CI supply chain is floating**	`requirements.txt`; workflow `npm install -g ...`; `actions/*@v4/v5`	Lock Python and CLI versions; pin Actions to reviewed full commit SHAs; update through deliberate dependency PRs. GitHub recommends full-SHA pinning for immutable third-party Action references. [1]

Severity	Finding	Evidence	Remediation
Medium	**Full merge suite is expensive enough that independent complete reproduction failed within ten minutes**	`tests/test_orchestration.py`, 108 tests; independent run	Add per-test timing, fast/integration labels, and a target runtime. Keep the full suite as final CI gate but make the common local gate fast.
Medium	**Backlog production substantially exceeds experimental throughput**	current materialization: ~80 ranked/unprocessed candidates versus 44 numbered ideas and only one current `PROBED` scrutiny state	Add charter backpressure. Nightly generation should skip when viable backlog exceeds a threshold or when expensive probes are already in flight.
Medium	**Actioner's self-modification path predates the stronger authority model**	`.github/workflows/actioner.yml:58–81`	Disable automatic improvement authoring until Patch 2b. The meta-loop should create proposal artifacts/issues only.
Low–medium	**Retired compatibility state is still live**	`_do_shortlist()` still appends `portfolio/ideas.csv`, `scout.py:636–637`; `_mean_score` / `scores_mean`; `results_v2` at `scout.py:1023` and workflow line 64	Remove in Patch 2c. Write one canonical result/run layout and one canonical ranking field.
Low–medium	**Results-workflow comments are outdated relative to current GitHub behavior**	`results-validate.yml:8–13` says GITHUB_TOKEN-created PRs do not trigger PR workflows	Current GitHub documentation says workflow-created PR `opened` / `synchronize` events can create approval-required workflow runs; update design/documentation accordingly. [2]
Low	**Architecture/design-history documentation remains mixed**	`REVAMP.md`, `docs/ARCHITECTURE.md`	Complete the planned split into current architecture, design history and live roadmap.

The security issue is architectural, not merely “use fewer secrets”

The results validator has already been improved substantially: trusted `main` code is checked out separately and result-branch contents are treated as data. That was exactly the right repair.

The other agent workflows have not yet made the same transition.

`actions/checkout@v4` persists authentication by default unless `persist-credentials:false` is set; the action's own documentation states this explicitly. [3] Meanwhile, `idea-pipeline` has `contents: write`, the Claude tool configuration permits `Bash(git:*)` and network-facing WebSearch/WebFetch, and the Codex CI path intentionally uses `--dangerously-bypass-approvals-and-sandbox`. A post-hoc filesystem scope check is not a security boundary against direct authenticated Git operations or network exfiltration.

The preferred pattern is:

```
secret-bearing model job
    checkout: read-only
    persist-credentials: false
    no repository-write token
    model writes local artifact
            ↓
deterministic validation
            ↓
small trusted write job
    exact allowlisted paths
    explicit GitHub permissions
    commit / push / issue creation
```

This is the infrastructure equivalent of the system's best scientific principle:

> **Models may advise and author candidates; deterministic machinery confers authority.**

This matters more once Issue comments become inputs. GitHub explicitly warns that fields such as issue bodies, PR bodies, titles and refs are potentially attacker-controlled and should not flow into executable shell syntax. [4]

I would also isolate the proposed **Codex secret write-back** into a tiny job that runs after all agent/untrusted-content processing and has no checkout or arbitrary shell execution beyond the narrowly defined credential update. A credential able to modify repository Actions secrets is itself a high-value administrative capability; it should never coexist with an LLM process.

Architecture review

Canonical state and experiment registry

I endorse Patch 2a, but I would change its language in one important respect.

`ideas/NNN/state.json` should **not be the "single machine truth."**

It should be:

> **the canonical materialized current-state view.**

The authority should remain the immutable evidence that produced it:

```
ledger events
decision receipts
contract/approval hashes
experiment registry
execution receipts
result receipts
```

Then:

```
those authorities
       ↓ deterministic materializer
ideas/NNN/state.json
```

A CI invariant should be:

> delete `state.json`, regenerate it, and obtain exactly the committed bytes.

That eliminates stale-state ambiguity without introducing another editable source of truth.

I would use a state schema approximately like:

```
{
  "schema_version": 1,
  "idea_id": "023",
  "charter": {
    "id": "isles24",
    "version": "..."
  },
  "source_candidate": "isles24-scout-...",
```

```
  "current_claim": {
    "claim_id": "...",
    "version": 3,
    "text": "..."
  },
  "status": "ACTIVE",
  "scrutiny": "PROBED",
  "role_epoch": "...",
  "active_probe": "023-stage0-census",
  "active_contract_hash": "...",
  "latest_result": null,
  "latest_interpretation": null,
  "pending_decisions": ["..."],
  "corrections": [],
  "materialization": {
    "generated_at": "...",
    "source_event_watermark": "...",
    "materializer_version": "..."
  }
}
```

The per-probe registry should remain separate because it answers a different question.

`state.json` asks:

> What is true about the idea **now**?

`registry.yaml` asks:

> What experiments exist, how do they depend on one another, and what artifacts bind each one?

I would not duplicate complete per-probe status in both.

A better registry node is:

```
id: outcome-census
claim_id: claim-023-v3
depends_on:
  - probe: synthetic-calibration
    condition: success
contract_hash: 349af5...
approval_receipt: decision-...
code_commit: d388e23...
consumes:
  - artifact: simulation_operating_characteristics.csv
```

```
    sha256: ...
produces:
  - artifact: per_stratum_summary.csv
  - artifact: split_manifest.csv
join_policy: all
status_source: execution-events
```

Critically, I would make **graph structure declarative but node status derived**. Hand-editing `status: COMPLETE` inside `registry.yaml` would recreate dual authority.

Branching probe DAGs need explicit semantics

Chains are easy. Branches introduce questions that DVC-like syntax alone does not answer.

Suppose:

The registry must distinguish:

- **data dependency**: D consumes C's artifact;
- **scientific prerequisite**: C is permitted only if A demonstrates X;
- **informational dependency**: B should be known but does not block C;
- **join policy**: D requires all upstream probes, any one, or a named logical predicate.

Otherwise a failed sibling can accidentally kill an entire idea.

I would require:

```
depends_on:
  all_of:
    - probe: A
      outcome: valid_complete
  artifacts:
    - probe: C
      output: foo.csv
```

and require stale propagation:

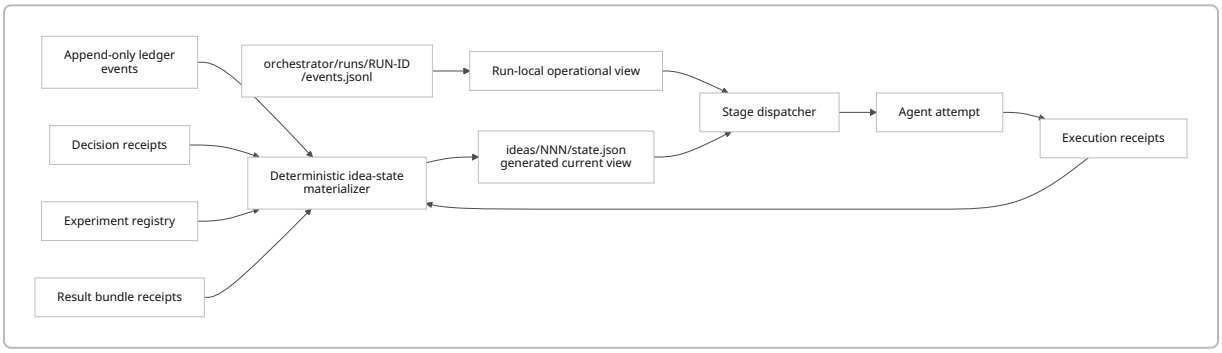
> if an upstream contract or consumed artifact hash changes, every downstream node consuming it becomes `STALE`, not silently `COMPLETE`.

Also, an idea-level status must not be inferred from one branch's terminal state. An idea can legitimately contain:

> one killed branch, one completed negative branch, and one still-active branch.

Run state should be separate from idea state

The architecture I recommend is:



This removes most legitimate reasons for a global mutable `orchestrator/state.json`.

A global file can still store UI convenience such as “last selected idea,” but scientific behavior should never depend on it.

Provenance should move inside the execution primitive

This is currently one of the largest mismatches between architecture and evidence.

`run_agent()` captures model information only in process-global `LAST_RUN` (`scout.py:391–462`). `_record_stage_provenance()` is a separate optional operation (`2369–2385`). Ordinary `_pipeline_stage()` invokes it, but debate explicitly returns before doing so (`2390–2395`), and specialized probe/interpret loops invoke `run_agent()` directly.

The consequence is visible in production: Idea 023's `stage_provenance.jsonl` does not provide a complete invocation history for the sequence the Round 4 packet describes—reconciliation, contract work, multiple code/review rounds, amendments and interpretation machinery. filecitefileturn0file0

Move attempt receipts into `run_agent()`:

```
def run_agent(..., context: RunContext) -> AgentResult:
    receipt = begin_execution_receipt(
        run_id=context.run_id,
```

```
        idea_id=context.idea_id,
        stage=stage,
        prompt_sha256=sha256(prompt_path),
        charter=context.charter,
        role_epoch=context.role_epoch,
        git_commit=head(),
    )

    for attempt_no, model in enumerate(models, start=1):
        result = invoke(...)
        receipt.append_attempt(
            attempt_no=attempt_no,
            family=agent,
            model=model,
            started_at=...,
            ended_at=...,
            exit_class=...,
            cli_version=...,
        )

    receipt.close(...)
    return result
```

Then separately write:

> artifact validated → promoted to canonical stage output.

That distinction matters because a model invocation succeeding is not the same as a scientific stage succeeding.

Charter isolation is better, but not complete

The recent identity/scoping work has solved the dangerous part: target charter resolution and score-bearing digest/portfolio context.

The remaining problem is policy.

Every non-blind prompt still receives the one global `docs/SCORING_RUBRIC.md` (`scout.py:197–201`), and the shared generation/schema machinery remains rooted in the original interpretability program.

So today:

> charter identity is properly separated;

but:

charter scientific grammar is not fully separated.

I would make the charter bundle:

```
charters/isles24/
    CHARTER.md
    generation_policy.md
    rubric.toml
    candidate.schema.json
```

while keeping shared:

```
keystone discipline
literature/source requirements
contract mechanism
negative-result semantics
human-gate rules
provenance
```

That is the right abstraction boundary.

There is also one small residual leak: `_digest_path()` at `scout.py:171–181` can still fall from a nonexistent named-charter digest to global `ledger_digest.md`. Make this fail/empty the same way the portfolio-brief fix now does.

Research-system design and public comparators

The system's identity is becoming clearer in comparison with other public work.

What to borrow—and what not to borrow

Kaimen's open-source Co-Scientist implementation has a more mature generic runtime architecture: a Supervisor dispatches work through a durable SQLite-backed task queue, model providers are abstracted, and benchmark runs are isolated from production session state. Its repository also stores bench runs, candidate outputs, match results and transcripts in explicit durable structures. [5]

The lesson to borrow is **not "switch to SQLite."** Your git-native/no-server constraint is legitimate.

The lesson is:

> **separate durable event/run state from the scientific artifacts those runs produce.**

Your per-run event directories plus derived state can achieve most of that without a service.

IDEAgent is substantially stronger than your scout at formalizing **quality-diversity evaluation**. It maintains ideation lineages, compares against prior/rejected ideas, ships explicit baselines, and defines Yield as a joint quality/diversity metric; its authors report materially higher Yield than their baselines across 32 CS topics. [6]

The important lesson is again evaluation, not architecture:

> Do not merely add diversity machinery. Establish a baseline, apply one intervention, and measure whether the set of usable distinct ideas improves.

ResearchAgent is structurally much simpler but has a cleaner literature substrate: it begins from Semantic Scholar paper IDs, retrieves related literature and knowledge entities, and iteratively generates/reviews/refines research problems. [7]

Your system is much stronger after ideation, but ResearchAgent points toward a future improvement you still lack:

> a normalized durable source registry that knows which paper supports which claim, rather than relying primarily on free-form agent prose and manifests.

Google's published Co-Scientist is especially relevant to the planned telemetry/meta work because the paper did not merely assume its agents helped: the authors report component ablations and analyses of Ranking, Evolution, Proximity and Meta-review behavior. [8]

That should be the standard for your optional stages:

> **an always-on component has not earned permanence merely because telemetry correlates with good outcomes.**

Blinding is right, but should be treated as information-flow control

I strongly support blinding the critic to sibling scores.

However, remove more than the explicit numeric field.

A critic can infer priority from:

> candidate ordering, backlog position, prose such as “top-ranked,” portfolio wording, filenames, or prior verdict context.

So define a **critic-context schema**, not just a text substitution:

```
ALLOW:
candidate scientific content
source evidence
```

```
charter
hard constraints
relevant factual context

DENY:
self-score
sibling scores
ranking
shortlist position
prior model preference
family identity
```

Similarly, “blind code reviewer to author family” should be understood as **label blinding**, not true blinding. Model-specific prose/style may still reveal provenance. That is okay; just do not overstate what the intervention accomplishes.

Diversity ordering

I would modify the planned order slightly.

Your current sequence is roughly:

> blinding → quotas → reseeding → possible selection pressure. filecite turn0file0

I prefer:

> **baseline measurement → blinding → randomized distant-context reseeding → soft quota/concentration control → selection pressure only if still needed.**

Reason: quota pressure can yield **cosmetic diversity**. A generator learns to label closely related ideas as different templates without actually exploring a different scientific neighborhood.

Reseeding changes the inputs to idea formation and therefore gives you a cleaner causal test.

A simple prospective design is:

```
same charter / same cycle format
      ↓
half generation slots: ordinary seed context
half generation slots: distant-context reseed
      ↓
blind evaluation
      ↓
compare usable-family diversity + survival
```

Only if collapse persists would I add an explicit concentration penalty.

Elo deferral is correct

I agree with deferring Elo/tournament selection.

Your bottleneck is currently **execution bandwidth**, not insufficient discrimination among hundreds of candidates.

Full pairwise tournament cost grows quadratically, and it would introduce judge-bias questions before you've established that present ranking is the limiting factor.

If you revisit pairwise selection, use it only around the decision boundary:

> top 4–6 finalists;

or:

> active/Swiss pairing;

or:

> one blinded comparative ranking pass.

Maintain an absolute viability floor. Elo should answer:

> A versus B?

It should never convert:

> two scientifically bad ideas

into:

> one winner.

Kaimen's implementation demonstrates that isolated tournament/bench machinery can be useful, but their benchmark system explicitly keeps bench state separate from production leaderboard state; that separation is worth copying if you ever add comparative ranking. [9]

Telemetry needs one additional metric

The proposed telemetry is useful:

> token cost, verdict distributions, review rounds, time-to-decision, diversity concentration, kill codes. filecite turn0file0

The one metric I would add is:

> **validated reviewer yield**.

For every critic/reviewer:

```
novel blocking findings introduced by that reviewer
        ↓
later confirmed / fixed / human-ratified
        ÷
review rounds or reviewer calls
```

Why this one?

A high agreement rate can mean excellent reviewers **or collusion/rubber-stamping**.

A high number of objections can mean incisiveness **or obstruction**.

Validated novel finding yield asks:

> Did this reviewer discover problems that mattered and survived independent adjudication?

Also track downstream reversal/calibration as a secondary measure:

> how often did a stage's recommendation later turn out wrong once stronger independent evidence arrived?

Those are much more diagnostic than raw agreement.

The interrogation channel needs a stricter authority boundary

“Answer/propose, never silently edit” is necessary but not sufficient.

The main abuse cases are:

- **review shopping**: repeatedly interrogate until one model produces the desired justification;
- **post-result contract drift**: ask a “clarifying” question after seeing results and smuggle a changed estimand into a proposal;
- **unratified-context leakage**: interrogation answer automatically enters later prompts and therefore influences decisions despite never being accepted;
- **prompt injection** through free-form Issue content;
- **authority laundering**: “the system explained why this is okay” is mistaken for “the contract permits it.”

Every interrogation should create a receipt:

```
question_id:
target_artifact_hash:
question_text:
results_visible_at_time: true|false
mode: EXPLANATION | CHANGE_PROPOSAL
answer_artifact:
```

And the rule should be:

> `EXPLANATION` may never alter downstream scientific context.
>
> `CHANGE_PROPOSAL` re-enters the exact review/gate path appropriate to the artifact being changed.

A proposed contract revision after results exposure should be explicitly marked post hoc and cannot retroactively change the interpretation rules for the already observed result.

The blackboard arbiter is plausible, but daily action count is the wrong safety primitive

A git-native issue blackboard can work within your no-server constraint.

But:

> “at most N actions/day”

is too crude.

One action could be:

> refresh a brief

while another is:

> launch a 100-case analysis.

Use category budgets:

Category	Example bound
Generation	candidate/stage-call cap
Literature/network	call/token cap
Repository mutation	PR/commit cap

Category	Example bound
Human attention	pending-decision/Issue cap
Compute	GPU/download/experiment cap
Scientific authority	**zero autonomous approvals**

The arbiter must dispatch only a finite enumerated action schema:

```
{
  "action": "RUN_STAGE",
  "idea_id": 23,
  "stage": "feasibility",
  "expected_state_hash": "...",
  "idempotency_key": "..."
}
```

Never:

```
"do whatever seems appropriate next"
```

Additional git-native failure modes include stale-blackboard TOCTOU decisions, issue replay, duplicate dispatch, starvation by cheap high-priority work, mutually blocked decisions, action amplification inside a stage, and prompt-injected Issue content.

I would implement the arbiter **after**, not during, the state/receipt work.

The meta-loop boundary can be made much simpler

The proposed semantic boundary—

> research heuristic changes yes, governance/gate changes no—

is harder to enforce than necessary. filecite turn0file0

The strongest mechanical rule is:

> **The meta-agent never modifies repository source at all.**

It may emit only:

```
evidence/meta_proposals/<id>.json
```

with a fixed schema.

A deterministic wrapper may then open an Issue from that proposal.

No checkout write credentials. No patching. No direct edits.

Then it does not matter whether a proposal concerns a prompt threshold or gate architecture: **all proposals remain advisory until the ordinary human/patch process handles them**.

That is much more enforceable than trying to classify the semantics of every edited line.

The third charter is scientifically interesting but unusually confounded

Using the system's own operating record to study multi-agent cognition creates several endogenous relationships:

> the system determines which ideas exist;
>
> the system determines which ones survive;
>
> the system determines which agent sees which information;
>
> the system's policies change during observation;
>
> the same model families may generate the hypothesis and judge the hypothesis;
>
> only survivors reach strong downstream evidence.

That creates selection bias, adaptive hypothesis formation, model-version confounding, survivor bias, non-independent observations and treatment contamination.

I would allow this charter only prospectively.

For any causal question:

1. Freeze the primary hypothesis and estimand.
2. Define the unit: cycle, candidate, review or stage.
3. Randomize the treatment prospectively.
4. Freeze model/family/version or stratify changes.
5. Freeze the evaluation script and judge before results.
6. Hold out future cycles; do not turn retrospective telemetry into causal evidence.
7. Cluster uncertainty by cycle/idea where observations share prompts/context.
8. Do not change the operating system in response to the experiment until the preregistered observation period is over.
9. Label historical analyses **hypothesis-generating only**.
10. Restrict claims to this operational environment unless reproduced elsewhere.

That would turn the self-reference from a weakness into an interesting methodological feature.

Roadmap

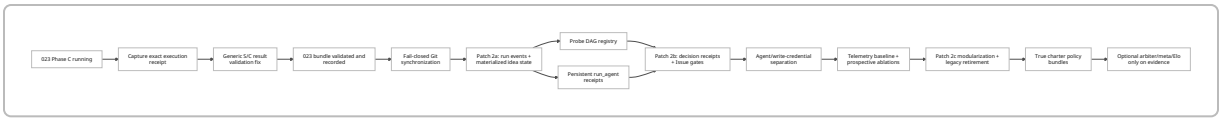
I would modify Claude's patch order more substantially than I expected before this audit.

Recommended sequence

Priority	Work	Approximate solo effort	Risk if deferred	Exit criterion
Immediate	**Capture and reconcile live 023 Phase-C invocation**	1–2 h	**Critical**	Exact running command, notebook/commit, Phase-S directory/hash and contract hash recorded
Immediate	**Make generic result transport accept 023 without changing its science**	4–8 h	**Critical**	Synthetic 023-C bundle passes; invalid C bundles fail; 004 regression still passes
Immediate	**Make Git synchronization fail closed**	2–4 h	High	No `pull --rebase ... || true`; failed rebase aborts and job fails without commit
Patch 2a	Per-idea materialized state + run-event separation	1–3 days	High	state regenerates deterministically; no scientific logic depends on active global cycle
Patch 2a	Probe DAG registry	1–2 days	Medium	DAG schema validates cycles, stale dependencies, joins and content hashes
Patch 2a	Persistent execution receipts inside `run_agent()`	1–2 days	High	Every agent invocation, including debate/probe/interpret/reconcile, has a receipt
Patch 2b foundation	Privilege separation for agent workflows	1–2 days	High	Model execution has no GitHub write credential; deterministic writer owns commits
Patch 2b	Decision receipts + Issue gates + interrogation rules	2–4 days	Medium–high	approval/revision/rejection binds actor + exact artifact/action hash

Priority	Work	Approximate solo effort	Risk if deferred	Exit criterion
Patch 2b	Locked reference-metric recompute	1 day	Medium	claim-critical reference metrics recomputed independently during verification
Evaluation pass	Baseline system telemetry + reviewer-yield metric	1–2 days	Medium	one stable report, no causal claims from observational associations
Research-side	Blinding + prospective reseeding experiment	1–2 cycles	Medium	measured effect on diversity/survival, not just more generated ideas
Patch 2c	Modularize `scout.py`; canonical results layout; remove CSV/aliases	3–5 days	Medium	one result/run layout, one ranking name, retired compatibility writes removed
Charter maturation	Charter-specific generation policy/ rubric/schema	2–3 days	Medium	genuinely different charter produces genuinely different research grammars
Later only	Arbiter	2–4 days	Low currently	justified by operator workload after receipts/ state stable
Later only	Meta-loop / third charter / Elo	Research experiment	Low currently	explicit evidence that current simpler system is bottlenecked

The timeline I would use is:



Generation backpressure should be added cheaply

The current repository already has roughly **80 viable** `SCOUT_ONLY` **entries** and 44 numbered ideas.

I would not run baseline scouting nightly merely because the cron exists.

One simple rule is sufficient:

```
if viable_backlog(charter) >= 20:
    print("Scout skipped: execution backlog above cap")
    return
```

The exact threshold is not sacred. The important principle is:

> generation rate should respond to research-execution capacity.

The project began because ideas were cheap and validity expensive. The automation should now embody that insight.

Regression and property-test plan

The next tests should emphasize invariants rather than “this example produced a file.”

Test	Purpose	Priority
`test_phase_c_bundle_validates_from_contract_spec`	Prevent another Idea-004-specific universal validator	**P0**
`test_phase_c_launcher_supplies_all_declared_dependencies`	Catch missing `--phase-s-dir` class of bug	**P0**
`test_result_manifest_path_cannot_escape_bundle`	Reject `../../...`, symlink and directory entries	**P0**
`test_failed_rebase_never_commits_or_pushes`	Regression for real conflict-marker incidents	**P0**
`test_state_rebuild_is_byte_identical`	Make committed state a verifiable materialization	**P0/2a**

Test	Purpose	Priority
`test_changing_active_cycle_cannot_change_idea_state_or_role_epoch`	Eliminate global-cycle confound	**2a**
`test_every_run_agent_call_emits_receipt_on_success_and_failure`	Close provenance bypasses	**2a**
`test_debate_and_probe_review_have_execution_receipts`	Explicit coverage of current custom paths	**2a**
`test_registry_rejects_cycle`	DAG correctness	**2a**
`test_upstream_hash_change_marks_consumers_stale`	DAG stale propagation	**2a**
`test_failed_sibling_does_not_kill_independent_branch`	Branch semantics	**2a**
`test_named_charter_never_receives_global_digest_fallback`	Finish framing isolation	**P1**
`test_changing_isles_rubric_cannot_change_baseline_rank`	Strong charter-isolation property	**P1**
`test_critic_prompt_contains_no_sibling_score_or_rank_information`	Proper blinding	**Research C**
`test_meta_agent_output_paths_are_proposal_only`	Mechanical meta-loop boundary	**2b**
`test_issue_command_parser_treats_body_as_data`	Prevent shell/ prompt authority injection	**2b**
`test_approve_receipt_invalid_after_artifact_hash_change`	Core gate invariant	**2b**
`test_duplicate_issue_command_is_idempotent`	Replay safety	**2b**
`test_invalid_tombstones_never_enter_scientific_context`	Preserve recent repair	**Regression**

Test	Purpose	Priority
`test_workflow_output_consumers_have_declared_producers`	Catch auto-PR-style workflow schema mismatch	**P1**

I would also add a tiny “architectural invariant” test module with properties such as:

```
def test_scientific_target_never_uses_active_charter():
    ...

def test_agent_execution_cannot_persist_git_credentials():
    ...

def test_all_result_phases_are_declared_in_probe_spec():
    ...

def test_all_required_outputs_are_machine_readable():
    ...
```

Test-suite structure

A useful merge model would be:

```
tests/unit/            target < 30 s
tests/invariants/      target < 60 s
tests/git_integration/ target < 3 min
tests/workflows/       target < 2 min
tests/slow/            nightly / explicit
```

The **full suite can remain the hard merge gate**, but the developer should be able to run the first two continuously.

Direct answers and limitations

State schema and registry

Keep them separate.

`state.json` should be a deterministic materialized present-tense view; `registry.yaml` should describe experiment graph structure and immutable artifact bindings. Do not duplicate editable probe status across them.

Add to state:

> schema/materializer version, source-event watermark, claim identity/version, charter/version, source candidate, pending decision IDs, active probe/contract pointer, latest result/interpretation, role epoch and correction/conflict flags.

Add to registry nodes:

> claim tested, semantic dependency type, contract hash/version, approval receipt, code commit, consumed artifact hashes, produced artifact definitions, join policy, invalidation semantics and supersession.

Branching requires explicit `all_of` / `any_of` /artifact dependencies, stale propagation and independence of sibling failures.

Most importantly:

> **state is not authority; it is a reproducible view of authority.**

Interrogation channel

The proposed rule is necessary but insufficient.

Require immutable interrogation receipts, distinguish `EXPLANATION` from `CHANGE_PROPOSAL`, prevent unratified responses from automatically entering downstream prompt context, and route every proposed change through the target artifact's ordinary review path.

Questions asked after result exposure must be marked as such. They cannot retroactively revise the preregistered interpretation of that result.

Arbiter safety

Use per-category budgets, not only a daily count.

Budget generation, tokens/network, repository writes, human-decision creation and real compute separately. Scientific approvals remain budget zero—human only.

Every dispatch needs an immutable target hash, expected-state precondition and idempotency key.

Do not allow the arbiter to convert free-form natural language directly into shell operations.

Diversity ordering

I would change it to:

> **measurement → score/rank blinding → randomized reseeding → soft concentration controls → selection pressure if still needed.**

I would front-load reseeding ahead of hard quotas because quota penalties can create nominal/template diversity without true scientific diversity.

Elo deferral

Agree. Strongly.

Execution is the current bottleneck, not fine discrimination among candidates.

If revisited, use capped finalist pairings, active/Swiss pairing or one blinded comparative pass. Maintain absolute viability gates and never compare scores across charters.

Telemetry

The proposed set is a good operational base but does not by itself detect reviewer collusion or usefulness.

Add **validated reviewer yield**:

> novel blocker introduced → later independently confirmed/fixed.

Also report downstream reversal/calibration.

Do not infer causality from natural survival correlations. Google's published Co-Scientist includes actual component ablations; that is the appropriate standard for claiming an agent/stage helps. [8]

Meta-loop boundary

Make the enforceable rule stronger and simpler:

> **A meta run has read-only access to the repository and may emit bytes only in a schema-validated meta-proposal artifact. It possesses no repository-write credential and cannot modify source, configuration, governance, tests, contracts or scientific artifacts.**

A deterministic non-agent wrapper may turn the artifact into an Issue.

Then there is no semantic ambiguity about “research heuristic versus governance code.”

Freeze list

Not sufficient in its present form.

The science-bearing freeze is good, but the current result validator is incompatible with Idea 023's phase/output vocabulary, and the committed launcher lacks its Phase-S dependency argument.

I would freeze:

> 023 contract, probe science/code, approval, data/split rules and execution commit.

But permit:

> **a narrowly reviewed, deterministic generic result-transport/validation fix that cannot change any scientific computation.**

Before that patch, create a synthetic Phase-C result fixture representing exactly what `run.py` will produce.

Because `scout.py` is monolithic, after 023 is safely recorded I would then resume the broader freeze and complete modularization.

Blind spots in the current plan

The largest ones are:

Result-system generalization. Idea 023 has already outrun the 004-era results ontology.

Execution provenance. The plan discusses increasingly subtle agent comparisons while the current recorder still misses whole classes of agent invocation.

Backpressure. There is an abundance of candidate supply and scarcity of experiment execution, yet generation remains scheduled.

True charter policy isolation. Identity/context scoping is good; rubric/schema/generation policy remain partly global.

Prospective evaluation. Telemetry is not enough. Optional stages should get randomized/controlled ablations.

Agent privilege separation. The planned blackboard and meta-loop expand the attack/authority surface before current model jobs have been separated from repository-write credentials.

The code-review readability checklist risks Goodharting. "≥1 assertion per transform" and a universal `<60 s smoke` should be interpreted scientifically, not mechanically. Prefer assertions on contract-critical transformations and a hermetic synthetic smoke target; do not reward meaningless assertions merely to satisfy a count.

Blinding leakage. Removing an explicit score or family label does not guarantee the same information isn't recoverable from prompt ordering, backlog position, provenance or style.

The referenced separate "Systems & Literature Survey and Meta-Charter Foundations" document was not present in the supplied snapshot/attachments I could inspect, so I cannot directly audit whether the survey

itself mischaracterizes individual papers. I evaluated the Round 4 plan's derived claims and independently checked the most relevant public systems instead.

Self-reference risk

The third charter can be scientifically worthwhile, but retrospective operational data should be treated as hypothesis generation.

Any causal study should prospectively freeze:

> hypothesis, estimand, unit of analysis, randomization, model/family assignment, judge, exclusions, analysis code/hash and observation window.

Use future holdout cycles.

Do not modify the operating system in response to interim results from that study.

Cluster observations appropriately because candidates within a cycle share prompts, memory, models and historical context.

And restrict conclusions accordingly:

> “in this scout configuration under these models and tasks”

is defensible;

> “multi-agent cognition works this way”

would require external replication.

Current GitHub-specific caveats

The trusted-code/results-data split in `results-validate.yml` is a major improvement. Continue that pattern.

For agent-bearing jobs, explicitly disable persisted checkout credentials unless a subsequent deterministic Git step needs them. The official checkout project documents that credentials are persisted by default in the versions you currently invoke. [3]

Use explicit least-privilege workflow permissions and full-SHA Action pins as part of Patch 2b/2c; GitHub recommends both as supply-chain hardening. [1]

And update the results workflow's GITHUB_TOKEN commentary. Current GitHub documentation says that PR `opened`, `synchronize` and `reopened` events caused by a workflow-created PR can produce **approval-required** workflow runs; the old blanket “won't trigger PR workflows” formulation is no longer accurate. [2]

Final assessment

The project is in a better position than it was during my previous reviews.

The important development is not that `scout.py` has accumulated more functionality. It is that **two materially different experiments have now stressed different parts of the architecture**:

> Idea 004 demonstrated a complete scientific path.
>
> Idea 023 is exposing whether that path was truly generalized or merely encoded around the first successful experiment.

That is exactly the test the system needed.

And the answer is mixed but encouraging:

The scientific workflow generalized.
The result infrastructure did not fully generalize yet.

Likewise:

Charter identity generalized.
Charter scientific policy has not fully generalized yet.

And:

Agent roles proliferated.
Execution provenance has not yet generalized across all those roles.

Those are tractable engineering problems.

The public systems reinforce rather than undermine the project's direction. Co-Scientist has stronger generic scheduling/storage and actual component ablations; IDEAgent has stronger quality-diversity evaluation; ResearchAgent has a cleaner structured literature substrate. [10] But none of those comparisons suggests that this project should imitate their entire architecture.

The thing worth protecting here is the part your scout does unusually well:

> **turn an abundant stream of plausible research ideas into a small number of bounded, auditable empirical questions while preserving exactly why the others died.**

That is now an actual research tool rather than an architectural thought experiment.

The highest-value next move is therefore **not more intelligence**.

It is to make four boundaries mechanically true:

> **run state is not scientific state;**
>
> **model output is not authority;**
>
> **charter identity implies charter-specific policy;**
>
> **a probe contract defines its own execution/result interface rather than inheriting assumptions from the previous probe.**

Once those are true—and once Idea 023 has safely landed through a genuinely generic result path—I would regard the project as having crossed from a highly sophisticated personal prototype into a credible, reusable research orchestration system.

---

1 https://docs.github.com/en/code-security/tutorials/secure-your-organization/protect-against-threats
https://docs.github.com/en/code-security/tutorials/secure-your-organization/protect-against-threats

2 https://docs.github.com/en/actions/concepts/security/github_token
https://docs.github.com/en/actions/concepts/security/github_token

3 https://github.com/actions/checkout
https://github.com/actions/checkout

4 https://docs.github.com/en/actions/concepts/security/script-injections
https://docs.github.com/en/actions/concepts/security/script-injections

5 9 10 https://github.com/Kaimen-Inc/Co-Scientist/blob/main/README.md
https://github.com/Kaimen-Inc/Co-Scientist/blob/main/README.md

6 https://github.com/declare-lab/IDEAgent
https://github.com/declare-lab/IDEAgent

7 https://github.com/JinheonBaek/ResearchAgent
https://github.com/JinheonBaek/ResearchAgent

8 https://www.nature.com/articles/s41586-026-10644-y
https://www.nature.com/articles/s41586-026-10644-y